

MCP Server Evaluation Guide

Overview

This document provides guidance on creating comprehensive evaluations for MCP servers. Evaluations test whether LLMs can effectively use your MCP server to answer realistic, complex questions using only the tools provided.

Quick Reference

Evaluation Requirements

- Create 10 human-readable questions
- Questions must be READ-ONLY, INDEPENDENT, NON-DESTRUCTIVE
- Each question requires multiple tool calls (potentially dozens)
- Answers must be single, verifiable values
- Answers must be STABLE (won't change over time)

Output Format

xml

```
<evaluation>
  <qa_pair>
    <question>Your question here</question>
    <answer>Single verifiable answer</answer>
  </qa_pair>
</evaluation>
```

Purpose of Evaluations

The measure of quality of an MCP server is NOT how well or comprehensively the server implements tools, but how well these implementations (input/output schemas, docstrings/descriptions, functionality) enable LLMs with no other context and access ONLY to the MCP servers to answer realistic and difficult questions.

Evaluation Overview

Create 10 human-readable questions requiring ONLY READ-ONLY, INDEPENDENT, NON-DESTRUCTIVE, and IDEMPOTENT operations to answer. Each question should be:

- Realistic
- Clear and concise
- Unambiguous
- Complex, requiring potentially dozens of tool calls or steps
- Answerable with a single, verifiable value that you identify in advance

Question Guidelines

Core Requirements

1. **Questions MUST be independent**
 - Each question should NOT depend on the answer to any other question
 - Should not assume prior write operations from processing another question
2. **Questions MUST require ONLY NON-DESTRUCTIVE AND IDEMPOTENT tool use**
 - Should not instruct or require modifying state to arrive at the correct answer
3. **Questions must be REALISTIC, CLEAR, CONCISE, and COMPLEX**
 - Must require another LLM to use multiple (potentially dozens of) tools or steps to answer

Complexity and Depth

4. **Questions must require deep exploration**
 - Consider multi-hop questions requiring multiple sub-questions and sequential tool calls
 - Each step should benefit from information found in previous questions
5. **Questions may require extensive paging**
 - May need paging through multiple pages of results
 - May require querying old data (1-2 years out-of-date) to find niche information
 - The questions must be DIFFICULT
6. **Questions must require deep understanding**
 - Rather than surface-level knowledge
 - May pose complex ideas as True/False questions requiring evidence
 - May use multiple-choice format where LLM must search different hypotheses
7. **Questions must not be solvable with straightforward keyword search**
 - Do not include specific keywords from the target content
 - Use synonyms, related concepts, or paraphrases
 - Require multiple searches, analyzing multiple related items, extracting context, then deriving the answer

Tool Testing

8. Questions should stress-test tool return values

- May elicit tools returning large JSON objects or lists, overwhelming the LLM
- Should require understanding multiple modalities of data:
 - IDs and names
 - Timestamps and datetimes (months, days, years, seconds)
 - File IDs, names, extensions, and mimetypes
 - URLs, GIDs, etc.
- Should probe the tool's ability to return all useful forms of data

9. Questions should **MOSTLY** reflect real human use cases

- The kinds of information retrieval tasks that HUMANS assisted by an LLM would care about

10. Questions may require dozens of tool calls

- This challenges LLMs with limited context
- Encourages MCP server tools to reduce information returned

11. Include ambiguous questions

- May be ambiguous OR require difficult decisions on which tools to call
- Force the LLM to potentially make mistakes or misinterpret
- Ensure that despite AMBIGUITY, there is STILL A SINGLE VERIFIABLE ANSWER

Stability

12. Questions must be designed so the answer **DOES NOT CHANGE**

- Do not ask questions that rely on "current state" which is dynamic
- For example, do not count:
 - Number of reactions to a post
 - Number of replies to a thread
 - Number of members in a channel

13. **DO NOT** let the MCP server **RESTRICT** the kinds of questions you create

- Create challenging and complex questions
- Some may not be solvable with the available MCP server tools
- Questions may require specific output formats (datetime vs. epoch time, JSON vs. MARKDOWN)
- Questions may require dozens of tool calls to complete

Answer Guidelines

Verification

1. Answers must be **VERIFIABLE** via direct string comparison

- If the answer can be re-written in many formats, clearly specify the output format in the QUESTION

- Examples: "Use YYYY/MM/DD.", "Respond True or False.", "Answer A, B, C, or D and nothing else."
- Answer should be a single VERIFIABLE value such as:
 - User ID, user name, display name, first name, last name
 - Channel ID, channel name
 - Message ID, string
 - URL, title
 - Numerical quantity
 - Timestamp, datetime
 - Boolean (for True/False questions)
 - Email address, phone number
 - File ID, file name, file extension
 - Multiple choice answer
- Answers must not require special formatting or complex, structured output
- Answer will be verified using DIRECT STRING COMPARISON

Readability

2. Answers should generally prefer HUMAN-READABLE formats

- Examples: names, first name, last name, datetime, file name, message string, URL, yes/no, true/false, a/b/c/d
- Rather than opaque IDs (though IDs are acceptable)
- The VAST MAJORITY of answers should be human-readable

Stability

3. Answers must be STABLE/STATIONARY

- Look at old content (e.g., conversations that have ended, projects that have launched, questions answered)
- Create QUESTIONS based on "closed" concepts that will always return the same answer
- Questions may ask to consider a fixed time window to insulate from non-stationary answers
- Rely on context UNLIKELY to change
- Example: if finding a paper name, be SPECIFIC enough so answer is not confused with papers published later

4. Answers must be CLEAR and UNAMBIGUOUS

- Questions must be designed so there is a single, clear answer
- Answer can be derived from using the MCP server tools

Diversity

5. Answers must be DIVERSE

- Answer should be a single VERIFIABLE value in diverse modalities and formats

- User concept: user ID, user name, display name, first name, last name, email address, phone number
 - Channel concept: channel ID, channel name, channel topic
 - Message concept: message ID, message string, timestamp, month, day, year
6. **Answers must NOT be complex structures**
- Not a list of values
 - Not a complex object
 - Not a list of IDs or strings
 - Not natural language text
 - UNLESS the answer can be straightforwardly verified using DIRECT STRING COMPARISON
 - And can be realistically reproduced
 - It should be unlikely that an LLM would return the same list in any other order or format

Evaluation Process

Step 1: Documentation Inspection

Read the documentation of the target API to understand:

- Available endpoints and functionality
- If ambiguity exists, fetch additional information from the web
- Parallelize this step AS MUCH AS POSSIBLE
- Ensure each subagent is ONLY examining documentation from the file system or on the web

Step 2: Tool Inspection

List the tools available in the MCP server:

- Inspect the MCP server directly
- Understand input/output schemas, docstrings, and descriptions
- WITHOUT calling the tools themselves at this stage

Step 3: Developing Understanding

Repeat steps 1 & 2 until you have a good understanding:

- Iterate multiple times
- Think about the kinds of tasks you want to create
- Refine your understanding
- At NO stage should you READ the code of the MCP server implementation itself

- Use your intuition and understanding to create reasonable, realistic, but VERY challenging tasks

Step 4: Read-Only Content Inspection

After understanding the API and tools, USE the MCP server tools:

- Inspect content using READ-ONLY and NON-DESTRUCTIVE operations ONLY
- Goal: identify specific content (e.g., users, channels, messages, projects, tasks) for creating realistic questions
- Should NOT call any tools that modify state
- Will NOT read the code of the MCP server implementation itself
- Parallelize this step with individual sub-agents pursuing independent explorations
- Ensure each subagent is only performing READ-ONLY, NON-DESTRUCTIVE, and IDEMPOTENT operations
- BE CAREFUL: SOME TOOLS may return LOTS OF DATA which would cause you to run out of CONTEXT
- Make INCREMENTAL, SMALL, AND TARGETED tool calls for exploration
- In all tool call requests, use the `limit` parameter to limit results (<10)
- Use pagination

Step 5: Task Generation

After inspecting the content, create 10 human-readable questions:

- An LLM should be able to answer these with the MCP server
- Follow all question and answer guidelines above

Output Format

Each QA pair consists of a question and an answer. The output should be an XML file with this structure:

xml

```
<evaluation>
  <qa_pair>
    <question>Find the project created in Q2 2024 with the highest number of completed tasks.
    What is the project name?</question>
    <answer>Website Redesign</answer>
  </qa_pair>
  <qa_pair>
    <question>Search for issues labeled as "bug" that were closed in March 2024. Which user
    closed the most issues? Provide their username.</question>
    <answer>sarah_dev</answer>
```

```
</qa_pair>
<qa_pair>
  <question>Look for pull requests that modified files in the /api directory and were merged
between January 1 and January 31, 2024. How many different contributors worked on these
PRs?</question>
  <answer>7</answer>
</qa_pair>
<qa_pair>
  <question>Find the repository with the most stars that was created before 2023. What is the
repository name?</question>
  <answer>data-pipeline</answer>
</qa_pair>
</evaluation>
```

Evaluation Examples

Good Questions

Example 1: Multi-hop question requiring deep exploration (GitHub MCP)

xml

```
<qa_pair>
  <question>Find the repository that was archived in Q3 2023 and had previously been the
most forked project in the organization. What was the primary programming language used in
that repository?</question>
  <answer>Python</answer>
</qa_pair>
```

This question is good because:

- Requires multiple searches to find archived repositories
- Needs to identify which had the most forks before archival
- Requires examining repository details for the language
- Answer is a simple, verifiable value
- Based on historical (closed) data that won't change

Example 2: Requires understanding context without keyword matching (Project Management MCP)

xml

```
<qa_pair>
  <question>Locate the initiative focused on improving customer onboarding that was
completed in late 2023. The project lead created a retrospective document after completion.
What was the lead's role title at that time?</question>
```

```
<answer>Product Manager</answer>
</qa_pair>
```

This question is good because:

- Doesn't use specific project name ("initiative focused on improving customer onboarding")
- Requires finding completed projects from specific timeframe
- Needs to identify the project lead and their role
- Requires understanding context from retrospective documents
- Answer is human-readable and stable
- Based on completed work (won't change)

Example 3: Complex aggregation requiring multiple steps (Issue Tracker MCP)

xml

```
<qa_pair>
  <question>Among all bugs reported in January 2024 that were marked as critical priority,
  which assignee resolved the highest percentage of their assigned bugs within 48 hours?
  Provide the assignee's username.</question>
  <answer>alex_eng</answer>
</qa_pair>
```

This question is good because:

- Requires filtering bugs by date, priority, and status
- Needs to group by assignee and calculate resolution rates
- Requires understanding timestamps to determine 48-hour windows
- Tests pagination (potentially many bugs to process)
- Answer is a single username
- Based on historical data from specific time period

Example 4: Requires synthesis across multiple data types (CRM MCP)

xml

```
<qa_pair>
  <question>Find the account that upgraded from the Starter to Enterprise plan in Q4 2023 and
  had the highest annual contract value. What industry does this account operate in?</question>
  <answer>Healthcare</answer>
</qa_pair>
```

This question is good because:

- Requires understanding subscription tier changes
- Needs to identify upgrade events in specific timeframe
- Requires comparing contract values

- Must access account industry information
- Answer is simple and verifiable
- Based on completed historical transactions

Poor Questions

Example 1: Answer changes over time

xml

```
<qa_pair>
  <question>How many open issues are currently assigned to the engineering
team?</question>
  <answer>47</answer>
</qa_pair>
```

This question is poor because:

- The answer will change as issues are created, closed, or reassigned
- Not based on stable/stationary data
- Relies on "current state" which is dynamic

Example 2: Too easy with keyword search

xml

```
<qa_pair>
  <question>Find the pull request with title "Add authentication feature" and tell me who created
it.</question>
  <answer>developer123</answer>
</qa_pair>
```

This question is poor because:

- Can be solved with a straightforward keyword search for exact title
- Doesn't require deep exploration or understanding
- No synthesis or analysis needed

Example 3: Ambiguous answer format

xml

```
<qa_pair>
  <question>List all the repositories that have Python as their primary language.</question>
  <answer>repo1, repo2, repo3, data-pipeline, ml-tools</answer>
</qa_pair>
```

This question is poor because:

- Answer is a list that could be returned in any order
- Difficult to verify with direct string comparison
- LLM might format differently (JSON array, comma-separated, newline-separated)
- Better to ask for a specific aggregate (count) or superlative (most stars)

Verification Process

After creating evaluations:

1. **Examine the XML file** to understand the schema
2. **Load each task instruction** and in parallel using the MCP server and tools, identify the correct answer by attempting to solve the task YOURSELF
3. **Flag any operations** that require WRITE or DESTRUCTIVE operations
4. **Accumulate all CORRECT answers** and replace any incorrect answers in the document
5. **Remove any <qa_pair>** that require WRITE or DESTRUCTIVE operations

Remember to parallelize solving tasks to avoid running out of context, then accumulate all answers and make changes to the file at the end.

Tips for Creating Quality Evaluations

1. **Think Hard and Plan Ahead** before generating tasks
 2. **Parallelize Where Opportunity Arises** to speed up the process and manage context
 3. **Focus on Realistic Use Cases** that humans would actually want to accomplish
 4. **Create Challenging Questions** that test the limits of the MCP server's capabilities
 5. **Ensure Stability** by using historical data and closed concepts
 6. **Verify Answers** by solving the questions yourself using the MCP server tools
 7. **Iterate and Refine** based on what you learn during the process
-

Running Evaluations

After creating your evaluation file, you can use the provided evaluation harness to test your MCP server.

Setup

1. **Install Dependencies**

bash

```
pip install -r scripts/requirements.txt
```

Or install manually:

bash

```
pip install anthropic mcp
```

2. Set API Key

bash

```
export ANTHROPIC_API_KEY=your_api_key_here
```

Evaluation File Format

Evaluation files use XML format with `<qa_pair>` elements:

xml

```
<evaluation>
  <qa_pair>
    <question>Find the project created in Q2 2024 with the highest number of completed tasks.
    What is the project name?</question>
    <answer>Website Redesign</answer>
  </qa_pair>
  <qa_pair>
    <question>Search for issues labeled as "bug" that were closed in March 2024. Which user
    closed the most issues? Provide their username.</question>
    <answer>sarah_dev</answer>
  </qa_pair>
</evaluation>
```

Running Evaluations

The evaluation script (`scripts/evaluation.py`) supports three transport types:

Important:

- **stdio transport:** The evaluation script automatically launches and manages the MCP server process for you. Do not run the server manually.
- **sse/http transports:** You must start the MCP server separately before running the evaluation. The script connects to the already-running server at the specified URL.

1. Local STDIO Server

For locally-run MCP servers (script launches the server automatically):

```
bash
python scripts/evaluation.py \
  -t stdio \
  -c python \
  -a my_mcp_server.py \
  evaluation.xml
```

With environment variables:

```
bash
python scripts/evaluation.py \
  -t stdio \
  -c python \
  -a my_mcp_server.py \
  -e API_KEY=abc123 \
  -e DEBUG=true \
  evaluation.xml
```

2. Server-Sent Events (SSE)

For SSE-based MCP servers (you must start the server first):

```
bash
python scripts/evaluation.py \
  -t sse \
  -u https://example.com/mcp \
  -H "Authorization: Bearer token123" \
  -H "X-Custom-Header: value" \
  evaluation.xml
```

3. HTTP (Streamable HTTP)

For HTTP-based MCP servers (you must start the server first):

```
bash
python scripts/evaluation.py \
  -t http \
  -u https://example.com/mcp \
  -H "Authorization: Bearer token123" \
  evaluation.xml
```

Command-Line Options

```
usage: evaluation.py [-h] [-t {stdio,sse,http}] [-m MODEL] [-c COMMAND]
                    [-a ARGS [ARGS ...]] [-e ENV [ENV ...]] [-u URL]
                    [-H HEADERS [HEADERS ...]] [-o OUTPUT]
                    eval_file
```

positional arguments:

eval_file Path to evaluation XML file

optional arguments:

-h, --help Show help message
-t, --transport Transport type: stdio, sse, or http (default: stdio)
-m, --model Claude model to use (default: claude-3-7-sonnet-20250219)
-o, --output Output file for report (default: print to stdout)

stdio options:

-c, --command Command to run MCP server (e.g., python, node)
-a, --args Arguments for the command (e.g., server.py)
-e, --env Environment variables in KEY=VALUE format

sse/http options:

-u, --url MCP server URL
-H, --header HTTP headers in 'Key: Value' format

Output

The evaluation script generates a detailed report including:

- **Summary Statistics:**
 - Accuracy (correct/total)
 - Average task duration
 - Average tool calls per task
 - Total tool calls
- **Per-Task Results:**
 - Prompt and expected response
 - Actual response from the agent
 - Whether the answer was correct (✓/✗)
 - Duration and tool call details
 - Agent's summary of its approach
 - Agent's feedback on the tools

Save Report to File

bash

`python scripts/evaluation.py \`

```
-t stdio \  
-c python \  
-a my_server.py \  
-o evaluation_report.md \  
evaluation.xml
```

Complete Example Workflow

Here's a complete example of creating and running an evaluation:

1. Create your evaluation file (`my_evaluation.xml`):

```
xml  
<evaluation>  
  <qa_pair>  
    <question>Find the user who created the most issues in January 2024. What is their  
username?</question>  
    <answer>alice_developer</answer>  
  </qa_pair>  
  <qa_pair>  
    <question>Among all pull requests merged in Q1 2024, which repository had the highest  
number? Provide the repository name.</question>  
    <answer>backend-api</answer>  
  </qa_pair>  
  <qa_pair>  
    <question>Find the project that was completed in December 2023 and had the longest  
duration from start to finish. How many days did it take?</question>  
    <answer>127</answer>  
  </qa_pair>  
</evaluation>
```

2. Install dependencies:

```
bash  
pip install -r scripts/requirements.txt  
export ANTHROPIC_API_KEY=your_api_key
```

3. Run evaluation:

```
bash  
python scripts/evaluation.py \  
-t stdio \  
-c python \  
-a github_mcp_server.py \  
-o evaluation_report.md
```

```
-e GITHUB_TOKEN=ghp_xxx \  
-o github_eval_report.md \  
my_evaluation.xml
```

4. **Review the report** in `github_eval_report.md` to:

- See which questions passed/failed
- Read the agent's feedback on your tools
- Identify areas for improvement
- Iterate on your MCP server design

Troubleshooting

Connection Errors

If you get connection errors:

- **STDIO**: Verify the command and arguments are correct
- **SSE/HTTP**: Check the URL is accessible and headers are correct
- Ensure any required API keys are set in environment variables or headers

Low Accuracy

If many evaluations fail:

- Review the agent's feedback for each task
- Check if tool descriptions are clear and comprehensive
- Verify input parameters are well-documented
- Consider whether tools return too much or too little data
- Ensure error messages are actionable

Timeout Issues

If tasks are timing out:

- Use a more capable model (e.g., `claude-3-7-sonnet-20250219`)
- Check if tools are returning too much data
- Verify pagination is working correctly
- Consider simplifying complex questions